



Higher-order Logic (HOL)

Classical Higher-Order Logic (HOL)

Expressivity	FOL	HOL	Example
Quantification over			
- Individuals	✓	✓	$\forall x p(f(x))$
- Functions	✗	✓	$\forall f p(f(a))$
- Predicates/Sets/Rels	✗	✓	$\forall p p(f(a))$
Unnamed			
- Functions	✗	✓	$(\lambda x x)$
- Predicates/Sets/Rels	✗	✓	$(\lambda x x \neq a)$
Statements about			
- Functions	✗	✓	<i>continuous</i> $(\lambda x x)$
- Predicates/Sets/Rels	✗	✓	<i>reflexive</i> $(=)$
Powerful abbreviations	✗	✓	<i>reflexive</i> $= \lambda r \forall x r(x, x)$

Classical Higher-Order Logic (HOL)

Expressivity	FOL	HOL	Example
Quantification over			
- Individuals	✓	✓	$\forall x_l p_{l \rightarrow o}(f_{l \rightarrow l}(x_l))$
- Functions	✗	✓	$\forall f_{l \rightarrow l} p_{l \rightarrow o}(f_{l \rightarrow o}(a_l))$
- Predicates/Sets/Rels	✗	✓	$\forall p_{l \rightarrow o} p_{l \rightarrow o}(f_{l \rightarrow l}(a_l))$
Unnamed			
- Functions	✗	✓	$(\lambda x_l x_l)$
- Predicates/Sets/Rels	✗	✓	$(\lambda x_{l \rightarrow l} x_{l \rightarrow l} \neq_{l \rightarrow l \rightarrow p} a)_l)$
Statements about			
- Functions	✗	✓	$continuous_{(l \rightarrow l) \rightarrow o}(\lambda x_l x_l)$
- Predicates/Sets/Rels	✗	✓	$reflexive_{(l \rightarrow l \rightarrow o) \rightarrow o}(=_{l \rightarrow l \rightarrow o})$
Powerful abbreviations	✗	✓	$reflexive_{(l \rightarrow l \rightarrow o) \rightarrow o} =$ $\lambda r_{(l \rightarrow l \rightarrow o)} \forall x_l r(x, x)$

Simple Types: Prevent Some Paradoxes and Inconsistencies

HOL: Syntax

Simple Types:

$$\alpha, \beta ::= \iota \mid o \mid (\alpha \rightarrow \beta)$$

(we may add further base types, e.g. μ)

HOL Language:

$$s, t ::= P_\alpha \mid x_\alpha \mid (\lambda x_\alpha s_\beta)_{\alpha \rightarrow \beta} \mid (s_{\alpha \rightarrow \beta} t_\alpha)_\beta \mid \\ (\neg_{o \rightarrow o} s_o) \mid ((\forall_{o \rightarrow o \rightarrow o} s_o) t_o) \mid \forall_{(\alpha \rightarrow o) \rightarrow o} (\lambda x_\alpha s_o)$$

constant symbols

variable symbols

lambda abstraction

application

negation

disjunction

universal quantification

Terms of type o : formulas

Other logical connectives can be defined, e.g. $\exists x s$ stands for $\neg \forall x \neg s$

Equality may also be defined: $s \doteq t$ stands for $\forall P (Ps \Rightarrow Pt)$
(but it is strongly recommended to add primitive equality!)

HOL: Syntax

Simple Types:

$$\alpha, \beta ::= \iota \mid o \mid (\alpha \rightarrow \beta)$$

(we may add further base types, e.g. μ)

HOL Language:

$$s, t ::= P_\alpha \mid x_\alpha \mid (\lambda x_\alpha s_\beta)_{\alpha \rightarrow \beta} \mid (s_{\alpha \rightarrow \beta} t_\alpha)_\beta \mid \\ (\neg_{o \rightarrow o} s_o) \mid ((\forall_{o \rightarrow o \rightarrow o} s_o) t_o) \mid \forall_{(\alpha \rightarrow o) \rightarrow o} (\lambda x_\alpha s_o)$$

constant symbols

variable symbols

lambda abstraction

application

negation

disjunction

universal quantification

Terms of type o : formulas

Other logical connectives can be defined, e.g. $\exists x s$ stands for $\neg \forall x \neg s$

Equality may also be defined: $s \doteq t$ stands for $\forall P (Ps \Rightarrow Pt)$
(but it is strongly recommended to add primitive equality!)

HOL: Syntax

Simple Types:

$$\alpha, \beta ::= \iota \mid o \mid (\alpha \rightarrow \beta)$$

(we may add further base types, e.g. μ)

HOL Language:

$$s, t ::= \boxed{P_\alpha} \mid x_\alpha \mid (\lambda x_\alpha s_\beta)_{\alpha \rightarrow \beta} \mid (s_{\alpha \rightarrow \beta} t_\alpha)_\beta \mid \\ (\neg_{o \rightarrow o} s_o) \mid ((\vee_{o \rightarrow o \rightarrow o} s_o) t_o) \mid \forall_{(\alpha \rightarrow o) \rightarrow o} (\lambda x_\alpha s_o)$$

constant symbols

variable symbols

lambda abstraction

application

negation

disjunction

universal quantification

Terms of type o : formulas

Other logical connectives can be defined, e.g. $\exists x s$ stands for $\neg \forall x \neg s$

Equality may also be defined: $s \doteq t$ stands for $\forall P (Ps \Rightarrow Pt)$

(but it is strongly recommended to add primitive equality!)

HOL: Syntax

Simple Types:

$$\alpha, \beta ::= \iota \mid o \mid (\alpha \rightarrow \beta)$$

(we may add further base types, e.g. μ)

HOL Language:

$$s, t ::= P_\alpha \mid \boxed{x_\alpha} \mid (\lambda x_\alpha s_\beta)_{\alpha \rightarrow \beta} \mid (s_{\alpha \rightarrow \beta} t_\alpha)_\beta \mid \\ (\neg_{o \rightarrow o} s_o) \mid ((\vee_{o \rightarrow o \rightarrow o} s_o) t_o) \mid \forall_{(\alpha \rightarrow o) \rightarrow o} (\lambda x_\alpha s_o)$$

constant symbols

variable symbols

lambda abstraction

application

negation

disjunction

universal quantification

Terms of type o : formulas

Other logical connectives can be defined, e.g. $\exists x s$ stands for $\neg \forall x \neg s$

Equality may also be defined: $s \doteq t$ stands for $\forall P (Ps \Rightarrow Pt)$

(but it is strongly recommended to add primitive equality!)

HOL: Syntax

Simple Types:

$$\alpha, \beta ::= \iota \mid o \mid (\alpha \rightarrow \beta)$$

(we may add further base types, e.g. μ)

HOL Language:

$$s, t ::= P_\alpha \mid x_\alpha \mid (\lambda x_\alpha s_\beta)_{\alpha \rightarrow \beta} \mid (s_{\alpha \rightarrow \beta} t_\alpha)_\beta \mid \\ (\neg_{o \rightarrow o} s_o) \mid ((\forall_{o \rightarrow o \rightarrow o} s_o) t_o) \mid \forall_{(\alpha \rightarrow o) \rightarrow o} (\lambda x_\alpha s_o)$$

constant symbols

variable symbols

lambda abstraction

application

negation

disjunction

universal quantification

Terms of type o : formulas

Other logical connectives can be defined, e.g. $\exists x s$ stands for $\neg \forall x \neg s$

Equality may also be defined: $s \doteq t$ stands for $\forall P (Ps \Rightarrow Pt)$
(but it is strongly recommended to add primitive equality!)

HOL: Syntax

Simple Types:

$$\alpha, \beta ::= \iota \mid o \mid (\alpha \rightarrow \beta)$$

(we may add further base types, e.g. μ)

HOL Language:

$$s, t ::= P_\alpha \mid x_\alpha \mid (\lambda x_\alpha s_\beta)_{\alpha \rightarrow \beta} \mid \boxed{(s_{\alpha \rightarrow \beta} t_\alpha)_\beta} \mid \\ (\neg_{o \rightarrow o} s_o) \mid ((\forall_{o \rightarrow o \rightarrow o} s_o) t_o) \mid \forall_{(\alpha \rightarrow o) \rightarrow o} (\lambda x_\alpha s_o)$$

constant symbols

variable symbols

lambda abstraction

application

negation

disjunction

universal quantification

Terms of type o : formulas

Other logical connectives can be defined, e.g. $\exists x s$ stands for $\neg \forall x \neg s$

Equality may also be defined: $s \doteq t$ stands for $\forall P (Ps \Rightarrow Pt)$

(but it is strongly recommended to add primitive equality!)

HOL: Syntax

Simple Types:

$$\alpha, \beta ::= \iota \mid o \mid (\alpha \rightarrow \beta)$$

(we may add further base types, e.g. μ)

HOL Language:

$$s, t ::= P_\alpha \mid x_\alpha \mid (\lambda x_\alpha s_\beta)_{\alpha \rightarrow \beta} \mid (s_{\alpha \rightarrow \beta} t_\alpha)_\beta \mid$$

$(\neg_{o \rightarrow o} s_o)$

 $\mid ((\forall_{o \rightarrow o \rightarrow o} s_o) t_o) \mid \forall_{(\alpha \rightarrow o) \rightarrow o} (\lambda x_\alpha s_o)$

constant symbols

variable symbols

lambda abstraction

application

negation

disjunction

universal quantification

Terms of type o : formulas

Other logical connectives can be defined, e.g. $\exists x s$ stands for $\neg \forall x \neg s$

Equality may also be defined: $s \doteq t$ stands for $\forall P (Ps \Rightarrow Pt)$
(but it is strongly recommended to add primitive equality!)

HOL: Syntax

Simple Types:

$$\alpha, \beta ::= \iota \mid o \mid (\alpha \rightarrow \beta)$$

(we may add further base types, e.g. μ)

HOL Language:

$$s, t ::= P_\alpha \mid x_\alpha \mid (\lambda x_\alpha s_\beta)_{\alpha \rightarrow \beta} \mid (s_{\alpha \rightarrow \beta} t_\alpha)_\beta \mid \\ (\neg_{o \rightarrow o} s_o) \mid ((\forall_{o \rightarrow o \rightarrow o} s_o) t_o) \mid \forall_{(\alpha \rightarrow o) \rightarrow o} (\lambda x_\alpha s_o)$$

constant symbols

variable symbols

lambda abstraction

application

negation

disjunction (we may use infix notation, omit types and brackets: $s \vee t$)

universal quantification

Terms of type o : formulas

Other logical connectives can be defined, e.g. $\exists x s$ stands for $\neg \forall x \neg s$

Equality may also be defined: $s \doteq t$ stands for $\forall P(Ps \Rightarrow Pt)$
(but it is strongly recommended to add primitive equality!)

HOL: Syntax

Simple Types:

$$\alpha, \beta ::= \iota \mid o \mid (\alpha \rightarrow \beta)$$

(we may add further base types, e.g. μ)

HOL Language:

$$s, t ::= P_\alpha \mid x_\alpha \mid (\lambda x_\alpha s_\beta)_{\alpha \rightarrow \beta} \mid (s_{\alpha \rightarrow \beta} t_\alpha)_\beta \mid$$

$$(\neg_{o \rightarrow o} s_o) \mid ((\forall_{o \rightarrow o \rightarrow o} s_o) t_o) \mid \boxed{\forall_{(\alpha \rightarrow o) \rightarrow o} (\lambda x_\alpha s_o)}$$

constant symbols

variable symbols

lambda abstraction

application

negation

disjunction

universal quantification

(we may use syntactical sugar: $\forall x_\alpha s$)

Terms of type o : formulas

Other logical connectives can be defined, e.g. $\exists x s$ stands for $\neg \forall x \neg s$

Equality may also be defined: $s \doteq t$ stands for $\forall P(Ps \Rightarrow Pt)$
(but it is strongly recommended to add primitive equality!)

HOL: Syntax

Simple Types:

$$\alpha, \beta ::= \iota \mid o \mid (\alpha \rightarrow \beta)$$

(we may add further base types, e.g. μ)

HOL Language:

$$s, t ::= P_\alpha \mid x_\alpha \mid (\lambda x_\alpha s_\beta)_{\alpha \rightarrow \beta} \mid (s_{\alpha \rightarrow \beta} t_\alpha)_\beta \mid \\ (\neg_{o \rightarrow o} s_o) \mid ((\forall_{o \rightarrow o \rightarrow o} s_o) t_o) \mid \forall_{(\alpha \rightarrow o) \rightarrow o} (\lambda x_\alpha s_o)$$

constant symbols

variable symbols

lambda abstraction

application

negation

disjunction

universal quantification

Terms of type o : formulas

Other logical connectives can be defined, e.g. $\exists x s$ stands for $\neg \forall x \neg s$

Equality may also be defined: $s \doteq t$ stands for $\forall P (Ps \Rightarrow Pt)$
(but it is strongly recommended to add primitive equality!)

HOL: Syntax

Simple Types:

$$\alpha, \beta ::= \iota \mid o \mid (\alpha \rightarrow \beta)$$

(we may add further base types, e.g. μ)

HOL Language:

$$s, t ::= P_\alpha \mid x_\alpha \mid (\lambda x_\alpha s_\beta)_{\alpha \rightarrow \beta} \mid (s_{\alpha \rightarrow \beta} t_\alpha)_\beta \mid \\ (\neg_{o \rightarrow o} s_o) \mid ((\forall_{o \rightarrow o \rightarrow o} s_o) t_o) \mid \forall_{(\alpha \rightarrow o) \rightarrow o} (\lambda x_\alpha s_o)$$

constant symbols

variable symbols

lambda abstraction

application

negation

disjunction

universal quantification

Terms of type o : formulas

Other logical connectives can be defined, e.g. $\exists x s$ stands for $\neg \forall x \neg s$

Equality may also be defined: $s \doteq t$ stands for $\forall P (Ps \Rightarrow Pt)$
(but it is strongly recommended to add primitive equality!)

HOL: Syntax

Simple Types:

$$\alpha, \beta ::= \iota \mid o \mid (\alpha \rightarrow \beta)$$

(we may add further base types, e.g. μ)

HOL Language:

$$s, t ::= P_\alpha \mid x_\alpha \mid (\lambda x_\alpha s_\beta)_{\alpha \rightarrow \beta} \mid (s_{\alpha \rightarrow \beta} t_\alpha)_\beta \mid \\ (\neg_{o \rightarrow o} s_o) \mid ((\forall_{o \rightarrow o \rightarrow o} s_o) t_o) \mid \forall_{(\alpha \rightarrow o) \rightarrow o} (\lambda x_\alpha s_o)$$

constant symbols

variable symbols

lambda abstraction

application

negation

disjunction

universal quantification

Terms of type o : formulas

Other logical connectives can be defined, e.g. $\exists x s$ stands for $\neg \forall x \neg s$

Equality may also be defined: $s \doteq t$ stands for $\forall P (Ps \Rightarrow Pt)$
(but it is strongly recommended to add primitive equality!)

α -conversion

is considered implicitly, e.g. $(\lambda x Px)$ is identified with $(\lambda y Py)$.

Substitution $[s/x]t$

of a term s_α for x_α in t_β is denoted by $[s/x]t$. We assume the bound variables of t avoid variable capture: $[y/x](\lambda y.Ryx)$ returns $(\lambda z.Rzy)$ and not $(\lambda y.Ryy)$

β -reduction and η -reduction

β -redex has the form $(\lambda x s)t$ and β -reduces to $[t/x]s$.

$$(\lambda x s)t \longrightarrow_\beta [t/x]s$$

η -redex has the form $(\lambda x sx)$ where x is not free in s ; it η -reduces to s .

$$(\lambda x sx) \longrightarrow_\eta s \quad (x \text{ is not free in } s)$$

$s \equiv_\beta t$ means s can be converted to t by β -reductions and expansions.

$s \equiv_{\beta\eta} t$ means s can be converted to t using both β and η .

Unique normalforms exist

For each $s_\alpha \in \mathcal{HOL}$ there is a unique β -normal form and a unique $\beta\eta$ -normal form.

α -conversion

is considered implicitly, e.g. $(\lambda x Px)$ is identified with $(\lambda y Py)$.

Substitution $[s/x]t$

of a term s_α for x_α in t_β is denoted by $[s/x]t$. We assume the bound variables of t avoid variable capture: $[y/x](\lambda y.Ryx)$ returns $(\lambda z.Rzy)$ and not $(\lambda y.Ryy)$

β -reduction and η -reduction

β -redex has the form $(\lambda x s)t$ and β -reduces to $[t/x]s$.

$$(\lambda x s)t \longrightarrow_\beta [t/x]s$$

η -redex has the form $(\lambda x sx)$ where x is not free in s ; it η -reduces to s .

$$(\lambda x sx) \longrightarrow_\eta s \quad (x \text{ is not free in } s)$$

$s \equiv_\beta t$ means s can be converted to t by β -reductions and expansions.

$s \equiv_{\beta\eta} t$ means s can be converted to t using both β and η .

Unique normalforms exist

For each $s_\alpha \in \mathcal{HOL}$ there is a unique β -normal form and a unique $\beta\eta$ -normal form.

α -conversion

is considered implicitly, e.g. $(\lambda x Px)$ is identified with $(\lambda y Py)$.

Substitution $([s/x]t)$

of a term s_α for x_α in t_β is denoted by $[s/x]t$. We assume the bound variables of t avoid variable capture: $[y/x](\lambda y.Ryx)$ returns $(\lambda z.Rzy)$ and not $(\lambda y.Ryy)$

β -reduction and η -reduction

β -redex has the form $(\lambda x s)t$ and β -reduces to $[t/x]s$.

$$(\lambda x s)t \longrightarrow_\beta [t/x]s$$

η -redex has the form $(\lambda x sx)$ where x is not free in s ; it η -reduces to s .

$$(\lambda x sx) \longrightarrow_\eta s \quad (x \text{ is not free in } s)$$

$s \equiv_\beta t$ means s can be converted to t by β -reductions and expansions.

$s \equiv_{\beta\eta} t$ means s can be converted to t using both β and η .

Unique normalforms exist

For each $s_\alpha \in \mathcal{HOL}$ there is a unique β -normal form and a unique $\beta\eta$ -normal form.

α -conversion

is considered implicitly, e.g. $(\lambda x Px)$ is identified with $(\lambda y Py)$.

Substitution $[s/x]t$

of a term s_α for x_α in t_β is denoted by $[s/x]t$. We assume the bound variables of t avoid variable capture: $[y/x](\lambda y.Ryx)$ returns $(\lambda z.Rzy)$ and not $(\lambda y.Ryy)$

β -reduction and η -reduction

β -redex has the form $(\lambda x s)t$ and β -reduces to $[t/x]s$.

$$(\lambda x s)t \longrightarrow_\beta [t/x]s$$

η -redex has the form $(\lambda x sx)$ where x is not free in s ; it η -reduces to s .

$$(\lambda x sx) \longrightarrow_\eta s \quad (x \text{ is not free in } s)$$

$s \equiv_\beta t$ means s can be converted to t by β -reductions and expansions.

$s \equiv_{\beta\eta} t$ means s can be converted to t using both β and η .

Unique normalforms exist

For each $s_\alpha \in \mathcal{HOL}$ there is a unique β -normal form and a unique $\beta\eta$ -normal form.

α -conversion

is considered implicitly, e.g. $(\lambda x Px)$ is identified with $(\lambda y Py)$.

Substitution $[s/x]t$

of a term s_α for x_α in t_β is denoted by $[s/x]t$. We assume the bound variables of t avoid variable capture: $[y/x](\lambda y.Ryx)$ returns $(\lambda z.Rzy)$ and not $(\lambda y.Ryy)$

β -reduction and η -reduction

β -redex has the form $(\lambda x s)t$ and β -reduces to $[t/x]s$.

$$(\lambda x s)t \longrightarrow_\beta [t/x]s$$

η -redex has the form $(\lambda x sx)$ where x is not free in s ; it η -reduces to s .

$$(\lambda x sx) \longrightarrow_\eta s \quad (x \text{ is not free in } s)$$

$s \equiv_\beta t$ means s can be converted to t by β -reductions and expansions.

$s \equiv_{\beta\eta} t$ means s can be converted to t using both β and η .

Unique normalforms exist

For each $s_\alpha \in \mathcal{HOL}$ there is a unique β -normal form and a unique $\beta\eta$ -normal form.

α -conversion

is considered implicitly, e.g. $(\lambda x Px)$ is identified with $(\lambda y Py)$.

Substitution $[s/x]t$

of a term s_α for x_α in t_β is denoted by $[s/x]t$. We assume the bound variables of t avoid variable capture: $[y/x](\lambda y.Ryx)$ returns $(\lambda z.Rzy)$ and not $(\lambda y.Ryy)$

β -reduction and η -reduction

β -redex has the form $(\lambda x s)t$ and β -reduces to $[t/x]s$.

$$(\lambda x s)t \longrightarrow_\beta [t/x]s$$

η -redex has the form $(\lambda x sx)$ where x is not free in s ; it η -reduces to s .

$$(\lambda x sx) \longrightarrow_\eta s \quad (x \text{ is not free in } s)$$

$s \equiv_\beta t$ means s can be converted to t by β -reductions and expansions.

$s \equiv_{\beta\eta} t$ means s can be converted to t using both β and η .

Unique normalforms exist

For each $s_\alpha \in \mathcal{HOL}$ there is a unique β -normal form and a unique $\beta\eta$ -normal form.

Semantics of HOL is (meanwhile) well understood

- ▶ Origin [Church, J.Symb.Log., 1940]
- ▶ Henkin-Semantics [Henkin, J.Symb.Log., 1950]
[Andrews, J.Symb.Log., 1971, 1972]
- ▶ Extensionality/Intensionality [BenzmüllerEtAl., J.Symb.Log., 2004]
[Muskens, J.Symb.Log., 2007]

HOL with Henkin-Semantics:

semi-decidable & compact & existence of countable models (like FOL)

Recommended Reading: [BenzmüllerMiller, HandbookHistoryOfLogic, Vol.9, 2014]

A Frame D

is a collection $\{D_\alpha\}_{\alpha \in T}$ of nonempty sets D_α , such that

- ▶ D_t can be chosen freely
- ▶ $D_o = \{T, F\}$ (for truth and falsehood), and
- ▶ $D_{\alpha \rightarrow \beta}$ are collections of functions mapping D_α into D_β

A Model M

for HOL is a tuple $M = \langle D, I \rangle$, where

- ▶ D is a frame;
- ▶ I is a family of typed interpretation functions mapping constant symbols p_α to appropriate elements of D_α , called the denotation of p_α ;
- ▶ the logical connectives $\neg$, $\vee$, and $\forall$ are always given the standard denotations;
- ▶ moreover, we assume that the domains $D_{\alpha \rightarrow \alpha \rightarrow o}$ contain the respective identity relations.

A Frame D

is a collection $\{D_\alpha\}_{\alpha \in T}$ of nonempty sets D_α , such that

- ▶ D_t can be chosen freely
- ▶ $D_o = \{T, F\}$ (for truth and falsehood), and
- ▶ $D_{\alpha \rightarrow \beta}$ are collections of functions mapping D_α into D_β

A Model M

for HOL is a tuple $M = \langle D, I \rangle$, where

- ▶ D is a frame;
- ▶ I is a family of typed interpretation functions mapping constant symbols p_α to appropriate elements of D_α , called the denotation of p_α ;
- ▶ the logical connectives $\neg$, $\vee$, and $\forall$ are always given the standard denotations;
- ▶ moreover, we assume that the domains $D_{\alpha \rightarrow \alpha \rightarrow o}$ contain the respective identity relations.

Standard and Henkin Models

In a standard model $M = \langle D, I \rangle$ we have

- ▶ $D_{\alpha \rightarrow \beta} = \{f \mid f : D_\alpha \longrightarrow D_\beta\}$ (for all types α, β)

In a Henkin model $M = \langle D, I \rangle$ we only require

- ▶ $D_{\alpha \rightarrow \beta} \subseteq \{f \mid f : D_\alpha \longrightarrow D_\beta\}$ (for all types α, β)
- ▶ the valuation function $\| \cdot \|^{M, g}$ from above is total (every term denotes)

Any standard model is obviously also a Henkin model.

We consider Henkin models in the remainder (for Henkin semantics we have semi-decidability, compactness and existence of countable models).

A Remark on Equality

Primitive equality is not needed in HOL, since equality can be defined:

$$\doteq := \lambda x_\alpha \lambda y_\alpha \forall p_{\alpha \rightarrow o}. px \rightarrow py \quad (\text{Leibniz})$$

$$\div := \lambda x_\alpha \lambda y_\alpha \forall q_{\alpha \rightarrow \alpha \rightarrow o}. \forall z_\alpha (qzz) \rightarrow qxy \quad (\text{Andrews})$$

For theoretical and pragmatic reasons the use of primitive equality is nevertheless highly recommended.

Standard and Henkin Models

In a standard model $M = \langle D, I \rangle$ we have

- ▶ $D_{\alpha \rightarrow \beta} = \{f \mid f : D_\alpha \longrightarrow D_\beta\}$ (for all types α, β)

In a Henkin model $M = \langle D, I \rangle$ we only require

- ▶ $D_{\alpha \rightarrow \beta} \subseteq \{f \mid f : D_\alpha \longrightarrow D_\beta\}$ (for all types α, β)
- ▶ the valuation function $\| \cdot \|^{M, g}$ from above is total (every term denotes)

Any standard model is obviously also a Henkin model.

We consider Henkin models in the remainder (for Henkin semantics we have semi-decidability, compactness and existence of countable models).

A Remark on Equality

Primitive equality is not needed in HOL, since equality can be defined:

$$\doteq := \lambda x_\alpha \lambda y_\alpha \forall p_{\alpha \rightarrow o}. px \rightarrow py \quad (\text{Leibniz})$$

$$\div := \lambda x_\alpha \lambda y_\alpha \forall q_{\alpha \rightarrow \alpha \rightarrow o}. \forall z_\alpha (qzz) \rightarrow qxy \quad (\text{Andrews})$$

For theoretical and pragmatic reasons the use of primitive equality is nevertheless highly recommended.

Variable Assignment

A variable assignment g maps variables x_α to elements in D_α .

$g[d/W]$ denotes the assignment that is identical to g , except for variable W , which is now mapped to d .

Interpretation/Value of a HOL term

The value $\|s_\alpha\|^{M,g}$ of a HOL term s_α on a model $M = \langle D, I \rangle$ under assignment g is an element $d \in D_\alpha$ defined in the following way:

1. $\|P_\alpha\|^{M,g} = I(P_\alpha)$
2. $\|x_\alpha\|^{M,g} = g(x_\alpha)$
3. $\|(s_{\alpha \rightarrow \beta} t_\alpha)_\beta\|^{M,g} = \|s_{\alpha \rightarrow \beta}\|^{M,g}(\|t_\alpha\|^{M,g})$
4. $\|(\lambda x_\alpha s_\beta)_{\alpha \rightarrow \beta}\|^{M,g}$ is the function f from D_α to D_β such that $f(d) = \|s_\beta\|^{M,g[d/x_\alpha]}$ for all $d \in D_\alpha$
5. $\|(\neg_{o \rightarrow o} s_o)_o\|^{M,g} = T$ iff $\|s_o\|^{M,g} = F$
6. $\|((\vee_{o \rightarrow o \rightarrow o} s_o) t_o)_o\|^{M,g} = T$ iff $\|s_o\|^{M,g} = T$ or $\|t_o\|^{M,g} = T$
7. $\|(\vee_{(\alpha \rightarrow o) \rightarrow o} (\lambda x_\alpha s_o))_o\|^{M,g} = T$ iff for all $d \in D_\alpha$ we have $\|s_o\|^{M,g[d/x_\alpha]} = T$

Variable Assignment

A variable assignment g maps variables x_α to elements in D_α .

$g[d/W]$ denotes the assignment that is identical to g , except for variable W , which is now mapped to d .

Interpretation/Value of a HOL term

The value $\|s_\alpha\|^{M,g}$ of a HOL term s_α on a model $M = \langle D, I \rangle$ under assignment g is an element $d \in D_\alpha$ defined in the following way:

1. $\|P_\alpha\|^{M,g} = I(P_\alpha)$
2. $\|x_\alpha\|^{M,g} = g(x_\alpha)$
3. $\|(s_{\alpha \rightarrow \beta} t_\alpha)_\beta\|^{M,g} = \|s_{\alpha \rightarrow \beta}\|^{M,g}(\|t_\alpha\|^{M,g})$
4. $\|(\lambda x_\alpha s_\beta)_{\alpha \rightarrow \beta}\|^{M,g}$ is the function f from D_α to D_β such that $f(d) = \|s_\beta\|^{M,g[d/x_\alpha]}$ for all $d \in D_\alpha$
5. $\|(\neg_{o \rightarrow o} s_o)_o\|^{M,g} = T$ iff $\|s_o\|^{M,g} = F$
6. $\|((\vee_{o \rightarrow o \rightarrow o} s_o) t_o)_o\|^{M,g} = T$ iff $\|s_o\|^{M,g} = T$ or $\|t_o\|^{M,g} = T$
7. $\|(\vee_{(\alpha \rightarrow o) \rightarrow o} (\lambda x_\alpha s_o))_o\|^{M,g} = T$ iff for all $d \in D_\alpha$ we have $\|s_o\|^{M,g[d/x_\alpha]} = T$

Variable Assignment

A variable assignment g maps variables x_α to elements in D_α .

$g[d/W]$ denotes the assignment that is identical to g , except for variable W , which is now mapped to d .

Interpretation/Value of a HOL term

The value $\|s_\alpha\|^{M,g}$ of a HOL term s_α on a model $M = \langle D, I \rangle$ under assignment g is an element $d \in D_\alpha$ defined in the following way:

1. $\|P_\alpha\|^{M,g} = I(P_\alpha)$
2. $\|x_\alpha\|^{M,g} = g(x_\alpha)$
3. $\|(s_{\alpha \rightarrow \beta} t_\alpha)_\beta\|^{M,g} = \|s_{\alpha \rightarrow \beta}\|^{M,g}(\|t_\alpha\|^{M,g})$
4. $\|(\lambda x_\alpha s_\beta)_{\alpha \rightarrow \beta}\|^{M,g}$ = the function f from D_α to D_β such that $f(d) = \|s_\beta\|^{M,g[d/x_\alpha]}$ for all $d \in D_\alpha$
5. $\|(\neg_{o \rightarrow o} s_o)_o\|^{M,g} = T$ iff $\|s_o\|^{M,g} = F$
6. $\|((\vee_{o \rightarrow o \rightarrow o} s_o) t_o)_o\|^{M,g} = T$ iff $\|s_o\|^{M,g} = T$ or $\|t_o\|^{M,g} = T$
7. $\|(\vee_{(\alpha \rightarrow o) \rightarrow o} (\lambda x_\alpha s_o))_o\|^{M,g} = T$ iff for all $d \in D_\alpha$ we have $\|s_o\|^{M,g[d/x_\alpha]} = T$

Variable Assignment

A variable assignment g maps variables x_α to elements in D_α .

$g[d/W]$ denotes the assignment that is identical to g , except for variable W , which is now mapped to d .

Interpretation/Value of a HOL term

The value $\|s_\alpha\|^{M,g}$ of a HOL term s_α on a model $M = \langle D, I \rangle$ under assignment g is an element $d \in D_\alpha$ defined in the following way:

1. $\|P_\alpha\|^{M,g} = I(P_\alpha)$
2. $\|x_\alpha\|^{M,g} = g(x_\alpha)$
3. $\|(s_{\alpha \rightarrow \beta} t_\alpha)_\beta\|^{M,g} = \|s_{\alpha \rightarrow \beta}\|^{M,g}(\|t_\alpha\|^{M,g})$
4. $\|(\lambda x_\alpha s_\beta)_{\alpha \rightarrow \beta}\|^{M,g}$ = the function f from D_α to D_β such that $f(d) = \|s_\beta\|^{M,g[d/x_\alpha]}$ for all $d \in D_\alpha$
5. $I(\neg_{o \rightarrow o}) = \text{not} \in D_{o \rightarrow o}$ such that: $\text{not}(F) = T$ and $\text{not}(T) = F$
6. $\|((\bigvee_{o \rightarrow o \rightarrow o} s_o) t_o)_o\|^{M,g} = T$ iff $\|s_o\|^{M,g} = T$ or $\|t_o\|^{M,g} = T$
7. $\|(\bigvee_{(\alpha \rightarrow o) \rightarrow o} (\lambda x_\alpha s_o))_o\|^{M,g} = T$ iff for all $d \in D_\alpha$ we have $\|s_o\|^{M,g[d/x_\alpha]} = T$

Variable Assignment

A variable assignment g maps variables x_α to elements in D_α .

$g[d/W]$ denotes the assignment that is identical to g , except for variable W , which is now mapped to d .

Interpretation/Value of a HOL term

The value $\|s_\alpha\|^{M,g}$ of a HOL term s_α on a model $M = \langle D, I \rangle$ under assignment g is an element $d \in D_\alpha$ defined in the following way:

1. $\|P_\alpha\|^{M,g} = I(P_\alpha)$
2. $\|x_\alpha\|^{M,g} = g(x_\alpha)$
3. $\|(s_{\alpha \rightarrow \beta} t_\alpha)_\beta\|^{M,g} = \|s_{\alpha \rightarrow \beta}\|^{M,g}(\|t_\alpha\|^{M,g})$
4. $\|(\lambda x_\alpha s_\beta)_{\alpha \rightarrow \beta}\|^{M,g}$ = the function f from D_α to D_β such that $f(d) = \|s_\beta\|^{M,g[d/x_\alpha]}$ for all $d \in D_\alpha$
5. $I(\neg_{o \rightarrow o}) = \text{not} \in D_{o \rightarrow o}$ such that: $\text{not}(F) = T$ and $\text{not}(T) = F$
6. $I(\vee_{o \rightarrow o \rightarrow o}) = \text{or} \in D_{o \rightarrow o \rightarrow o}$ such that: $\text{or}(a, b) = T$ iff $a = T$ or $b = T$
7. $\|(\vee_{(\alpha \rightarrow o) \rightarrow o} (\lambda x_\alpha s_o))_o\|^{M,g} = T$ iff for all $d \in D_\alpha$ we have $\|s_o\|^{M,g[d/x_\alpha]} = T$

Variable Assignment

A variable assignment g maps variables x_α to elements in D_α .

$g[d/W]$ denotes the assignment that is identical to g , except for variable W , which is now mapped to d .

Interpretation/Value of a HOL term

The value $\|s_\alpha\|^{M,g}$ of a HOL term s_α on a model $M = \langle D, I \rangle$ under assignment g is an element $d \in D_\alpha$ defined in the following way:

1. $\|P_\alpha\|^{M,g} = I(P_\alpha)$
2. $\|x_\alpha\|^{M,g} = g(x_\alpha)$
3. $\|(s_{\alpha \rightarrow \beta} t_\alpha)_\beta\|^{M,g} = \|s_{\alpha \rightarrow \beta}\|^{M,g}(\|t_\alpha\|^{M,g})$
4. $\|(\lambda x_\alpha s_\beta)_{\alpha \rightarrow \beta}\|^{M,g} = \text{the function } f \text{ from } D_\alpha \text{ to } D_\beta \text{ such that } f(d) = \|s_\beta\|^{M,g[d/x_\alpha]} \text{ for all } d \in D_\alpha$
5. $I(\neg_{o \rightarrow o}) = \text{not} \in D_{o \rightarrow o}$ such that: $\text{not}(F) = T$ and $\text{not}(T) = F$
6. $I(\vee_{o \rightarrow o \rightarrow o}) = \text{or} \in D_{o \rightarrow o \rightarrow o}$ such that: $\text{or}(a, b) = T$ iff $a = T$ or $b = T$
7. $I(\forall_{(\alpha \rightarrow o) \rightarrow o}) = \text{All} \in D_{(\alpha \rightarrow o) \rightarrow o}$ such that: for $p \in D_{\alpha \rightarrow o}$ we have $\text{All}(p) = T$ iff $p(d) = T$ for all $d \in D_\alpha$

Truth in model / Validity

A formula s_o is true in model M under assignment g if and only if $\|s_o\|^M g = T$; this is also denoted as $M, g \models s_o$.

A formula s_o is called valid in M if and only if $M, g \models s_o$ for all assignments g ; this is also denoted as $M \models s_o$.

A formula s_o is called valid, which we denote by $\models s_o$, if and only if $M \models s_o$ for all M .

We define $\models \varphi$, where φ is a set of HOL formulas, if and only if $\models s$ for all $s \in \varphi$.

Logical Consequence

Let φ and ψ be set of HOL formulas. We define

$$\varphi \models \psi \quad (\text{logical consequence})$$

if and only if for each model M we have:

$$M \models \varphi \quad \text{implies} \quad M \models \psi$$

HOL: Semantics

Truth in model / Validity

A formula s_o is true in model M under assignment g if and only if $\|s_o\|^M_g = T$; this is also denoted as $M, g \models s_o$.

A formula s_o is called valid in M if and only if $M, g \models s_o$ for all assignments g ; this is also denoted as $M \models s_o$.

A formula s_o is called valid, which we denote by $\models s_o$, if and only if $M \models s_o$ for all M .

We define $\models \varphi$, where φ is a set of HOL formulas, if and only if $\models s$ for all $s \in \varphi$.

Logical Consequence

Let φ and ψ be set of HOL formulas. We define

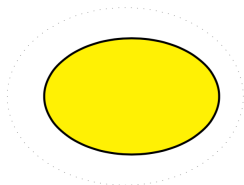
$$\varphi \models \psi \quad (\text{logical consequence})$$

if and only if for each model M we have:

$$M \models \varphi \quad \text{implies} \quad M \models \psi$$

HOL: Semantics

(see [BenzmüllerBrownKohlhase, J.Symb.Log., 2004] and [BenzmüllerBrownKohlhase, LMCS, 2009])



Standard Models $\mathfrak{M}(\Sigma)$

■ Idea of Standard Semantics:

$\iota \longrightarrow \mathcal{D}_\iota$ (choose)

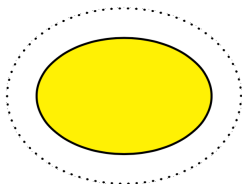
$\circ \longrightarrow \mathcal{D}_\circ = \{\mathbf{T}, \mathbf{F}\}$ (fixed)

$(\alpha \rightarrow \beta) \longrightarrow$

$\mathcal{D}_{\alpha \rightarrow \beta} = \mathcal{F}(\mathcal{D}_\alpha, \mathcal{D}_\beta)$ (fixed)

HOL: Semantics

(see [BenzmüllerBrownKohlhase, J.Symb.Log., 2004] and [BenzmüllerBrownKohlhase, LMCS, 2009])



Standard Models $\mathfrak{M}(\Sigma)$

Idea of Standard Semantics:

$\iota \longrightarrow \mathcal{D}_\iota$ (choose)

$\circ \longrightarrow \mathcal{D}_\circ = \{\mathbf{T}, \mathbf{F}\}$ (fixed)

$(\alpha \rightarrow \beta) \longrightarrow$
 $\mathcal{D}_{\alpha \rightarrow \beta} = \mathcal{F}(\mathcal{D}_\alpha, \mathcal{D}_\beta)$ (fixed)

Henkin's Generalization:

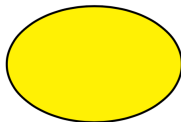
$\mathcal{D}_{\alpha \rightarrow \beta} \subseteq \mathcal{F}(\mathcal{D}_\alpha, \mathcal{D}_\beta)$ (choose)

but elements are still functions!

[Henkin-50]

HOL: Semantics

(see [BenzmüllerBrownKohlhase, J.Symb.Log., 2004] and [BenzmüllerBrownKohlhase, LMCS, 2009])



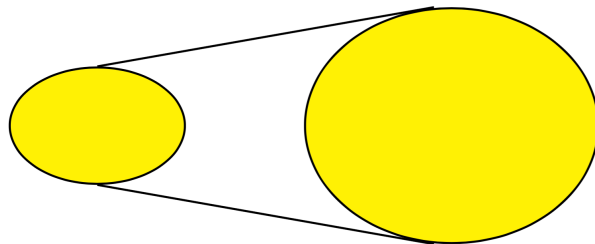
Standard Models $\mathfrak{M}(\Sigma)$

choose: $\mathcal{D}_i$

fixed: $\mathcal{D}_o, \mathcal{D}_{\alpha \rightarrow \beta}$, functions

HOL: Semantics

(see [BenzmüllerBrownKohlhase, J.Symb.Log., 2004] and [BenzmüllerBrownKohlhase, LMCS, 2009])



Standard Models $\mathfrak{SI}(\Sigma)$

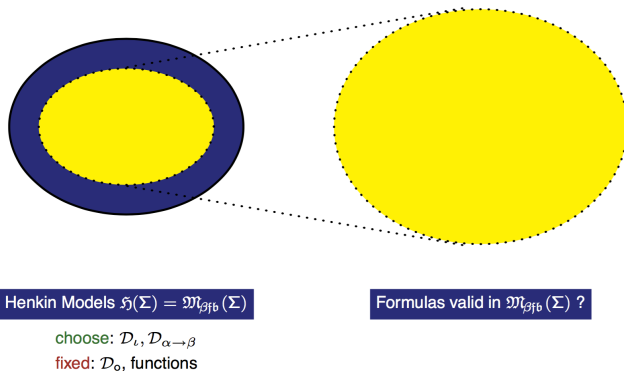
Formulas valid in $\mathfrak{SI}(\Sigma)$

choose: $\mathcal{D}_i$

fixed: $\mathcal{D}_o, \mathcal{D}_{\alpha \rightarrow \beta}$, functions

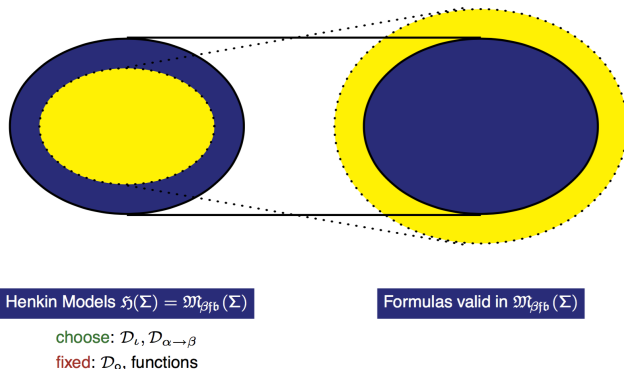
HOL: Semantics

(see [BenzmüllerBrownKohlhase, J.Symb.Log., 2004] and [BenzmüllerBrownKohlhase, LMCS, 2009])



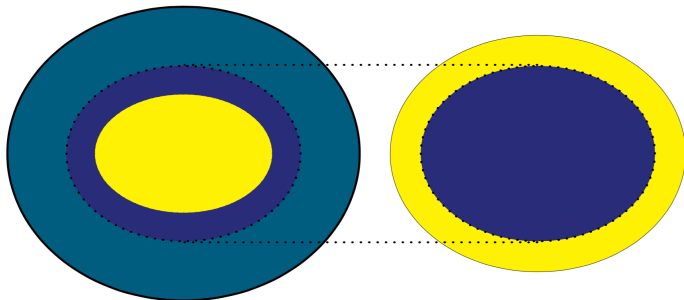
HOL: Semantics

(see [BenzmüllerBrownKohlhase, J.Symb.Log., 2004] and [BenzmüllerBrownKohlhase, LMCS, 2009])



HOL: Semantics

(see [BenzmüllerBrownKohlhase, J.Symb.Log., 2004] and [BenzmüllerBrownKohlhase, LMCS, 2009])



Non-Extensional Models $\mathfrak{M}_\beta(\Sigma)$

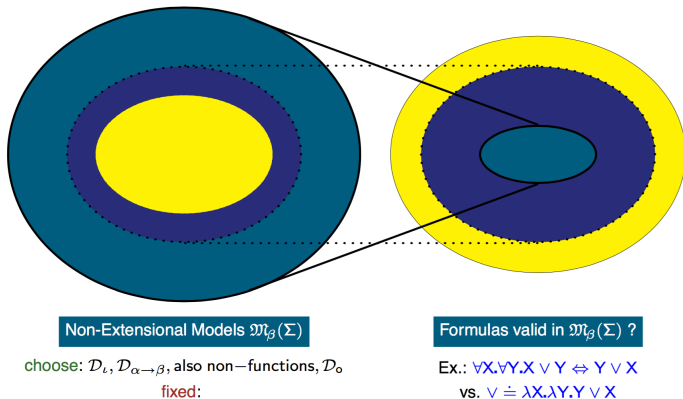
choose: $\mathcal{D}_\iota, \mathcal{D}_{\alpha \rightarrow \beta}$, also non-functions, $\mathcal{D}_o$

fixed:

Formulas valid in $\mathfrak{M}_\beta(\Sigma)$?

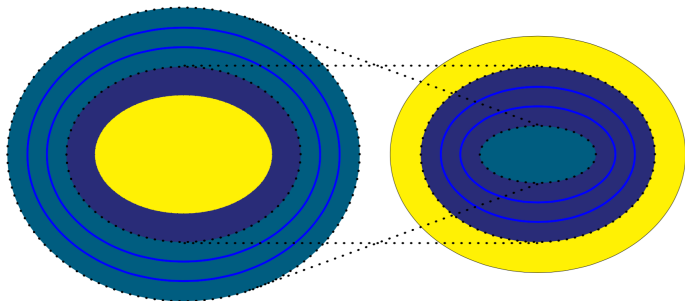
HOL: Semantics

(see [BenzmüllerBrownKohlhase, J.Symb.Log., 2004] and [BenzmüllerBrownKohlhase, LMCS, 2009])



HOL: Semantics

(see [BenzmüllerBrownKohlhase, J.Symb.Log., 2004] and [BenzmüllerBrownKohlhase, LMCS, 2009])



We additionally studied different model classes with 'varying degrees of extensionality'

$$\forall X. \forall Y. X \vee Y \Leftrightarrow Y \vee X$$

$$\forall X. \forall Y. X \vee Y \doteq Y \vee X$$

$$\lambda X. \lambda Y. X \vee Y \doteq \lambda X. \lambda Y. Y \vee X$$

$$\vee \doteq \lambda X. \lambda Y. Y \vee X$$